

Create a Utility Host & Build an AWS ParallelCluster

If you want to set up your own Linux based AWS EC2 instance to run Land Data Assimilation (DA) v3 container, follow section1, “**Create a Utility Host**” section of this document. If you already have a Linux-based utility host to run the Land DA v3 container-based workflow, you can directly go to section 2, “**Build the Utilities: Packer and the Amazon Machine Image (AMI)**”.

An AWS Linux utility host is identical to a login node on High Performance Computing (HPC) systems. It is created using the Amazon EC2 service and runs a Linux operating system such as Amazon Linux2 or Ubuntu. The utility host acts as an entry point to connect to AWS ParallelCluster environments. The Land DA v3 workflow can be run on AWS using AWS ParallelCluster. For that, you need to install the following prerequisites on the utility host (t3.micro or similar free tier eligible) you just created:

- **HashiCorp Packer 1.15:** An opensource tool from HashiCorp to create AWS machine image
- **AWS ParallelCluster client version 3.13.2:** An opensource tool from AWS to create and manage HPC clusters in cloud

Make sure the utility host is deployed in the same subnet that you plan to use for your AWS ParallelCluster so that network connectivity works correctly.

All UFS-WM applications and components have been tested in the AWS region “**us-east-1 (N. Virginia)**”. Functionality of these applications in other AWS regions is not guaranteed.

A paid AWS account is required to proceed with Land DA v3 container-based test run.

1. Create a Utility Host (i.e., an AWS instance)

Navigate to the AWS login page and login

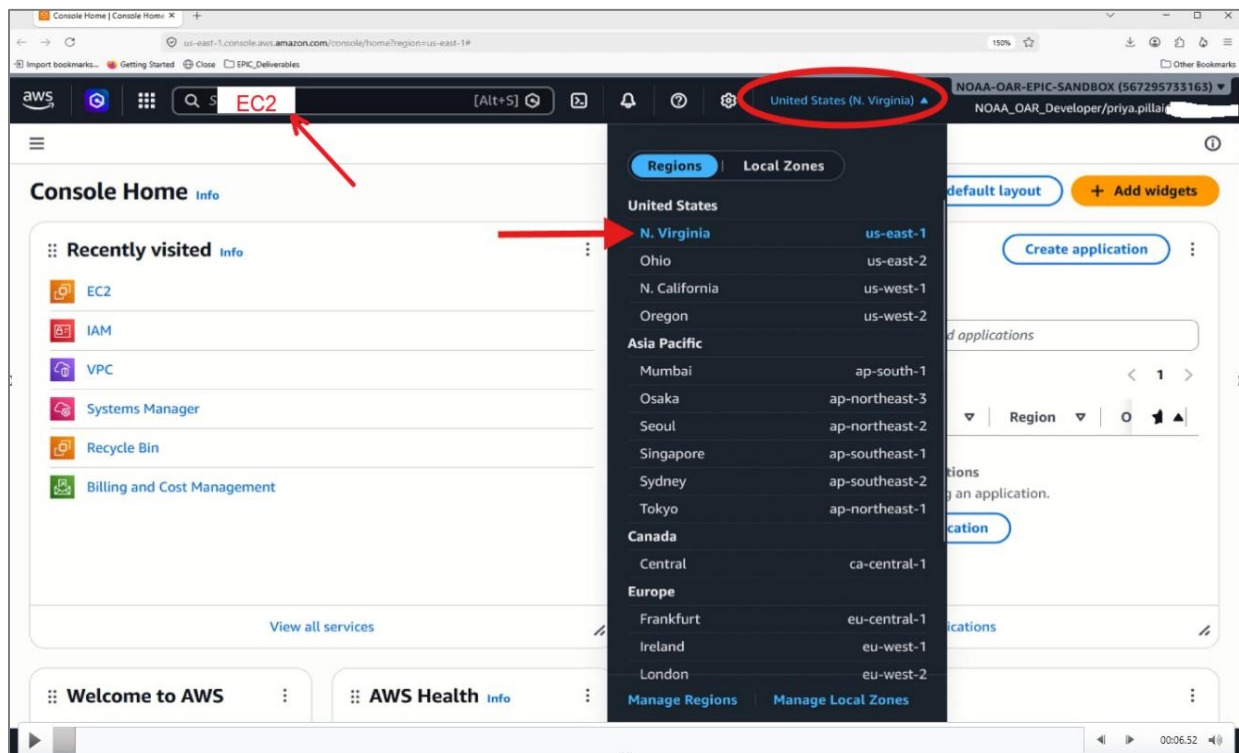
(a) If you are using **a personal AWS account**

You will need a paid AWS account to run Land DA experiments. Login at <https://console.aws.amazon.com/>, or sign up if you do not already have an account. This is ideal for individual learning & experimentation and is fully managed by the user. You should follow this “**Create a Utility Host and Build an AWS ParallelCluster**” guide. Once it is built, click on its “**instance-id**” to connect to the cluster.

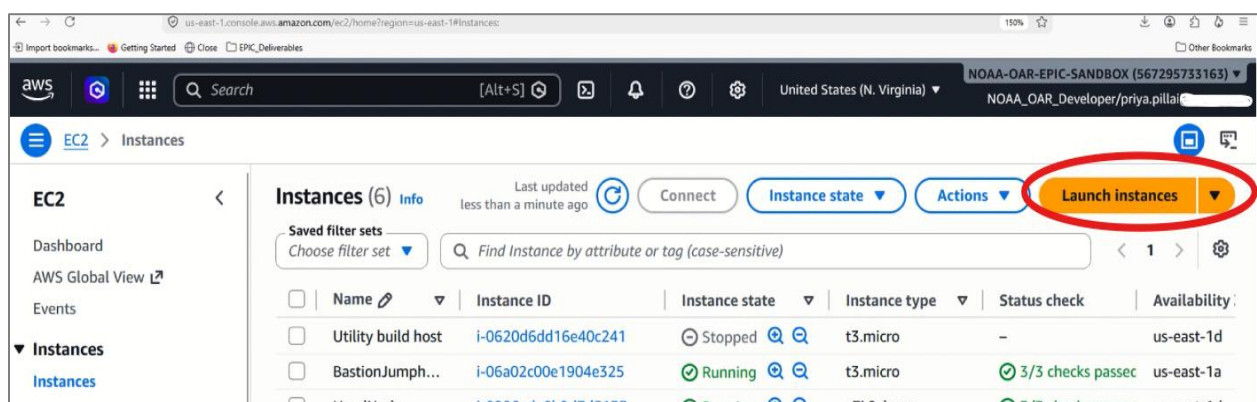
- (b) If you are using an **organization/institution provisioned AWS account (e.g. NOAA)**:
Login through the **AWS SSO URL** shared by your cloud administrator. This account is governed by institutional policies, permissions, & security controls. You may need to coordinate with your cloud admin to build the cluster. Once the cluster is created, your cloud admin will share an “**instance-id**” for you to connect to it.

Navigate to AWS console and start creating an AWS EC2 instance

Once logged in, navigate to the AWS Console and select region to **United States (N. Virginia)**. Search for “**EC2**” in the search bar to open the **AWS EC2 service page**.



On this page, click the “**instances**” button to begin creating an AWS EC2 instance. Click on “**Launch instance**” button



Proceed to fill in the “Launch an instance” page

- **Name and tags:** Create a name of your choice (“create_uhost-tutorial” is used in this documentation)
- **Application and OS Image:**
 - **AMI:** Amazon Linux
 - **Architecture:** 64-bit (x86)

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

e.g. My Web Server

Add additional tags

Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Instance type [Info](#) | [Get advice](#)

Instance type

Summary

Number of instances [Info](#)

1

Software Image (AMI)

-

Virtual server type (instance type)

-

Firewall (security group)

-

Storage (volumes)

-

Cancel

Launch instance

[Preview code](#)

- **Instance type:** t3.micro

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose [Browse more AMIs](#).

Search our full catalog including 1000s of application and OS images

Recents | My AMIs | **Quick Start**

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

[Browse more AMIs](#)

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 kernel-6.18 AMI

ami-08f44e8eca9095668 (64-bit (x86), uefi-preferred) / ami-0bba2fc7fccad71a7 (64-bit (Arm), uefi)

Free tier eligible

Virtualization: hvm ENA enabled: true Root device type: ebs

Description

Amazon Linux 2023 (kernel-6.18) is a modern, general purpose Linux-based OS that will be supported until June

Summary

Number of instances [Info](#)

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.12....[read more](#)

ami-08f44e8eca9095668

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Storage (volumes)

1 volume(s) - 8 GiB

Cancel

Launch instance

[Preview code](#)

Important Note: Users can connect to AWS EC2 instances using either SSH or an AWS System Manager Session (SSM) Session Manager. SSH requires opening inbound port 22 in the instance's security group and using an SSH key pair. For some organizations, security policies or firewalls do not allow port 22 to be opened, so users instead rely on the SSM Agent installed on the EC2 instance and AWS SSM Session Manager to establish a secure tunnel without exposing any inbound ports. Therefore, users need to choose between SSH and SSM Session Manager. In this tutorial, we use SSM Session Manager instead of traditional SSH.

Create Key Pair for SSH connection

You need to set up an SSH keypair only if you plan to use SSH-based connection. You can **skip this step** if you already have a key pair and it shows up in the AWS Public key drop-down list. Click **"Create key pair"** and fill in or confirm the following fields. After creating the keypair, you must allow SSH traffic in the security group (port 22). No IAM role changes are needed for SSH.

- **Key pair name:** Create_UHost
- **Key pair type:** RSA or ED25519
You can choose any key pair type. The difference is in the encryption algorithm used.
- **Private key file format:** .pem
- Click **"Create key pair"**

Create key pair

Key pair name
Key pairs allow you to connect to your instance securely.

create-uhost-tutorial-key

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair

Private key file format

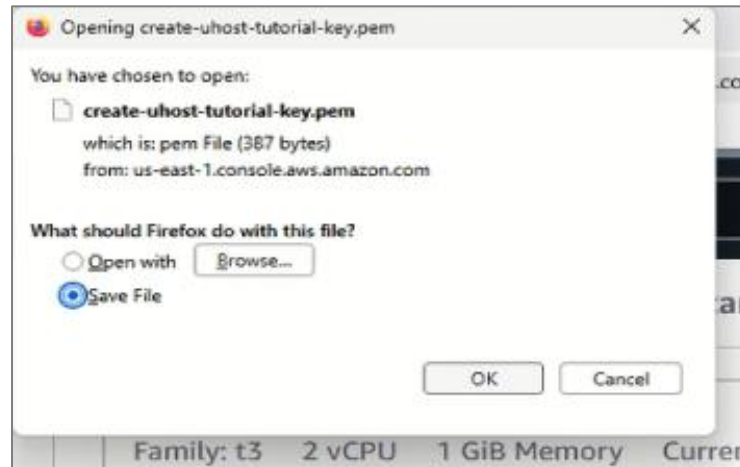
☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

⚠ When prompted, store the private key in a secure and accessible location on your computer. You will need it later to connect to your instance. [Learn more](#)

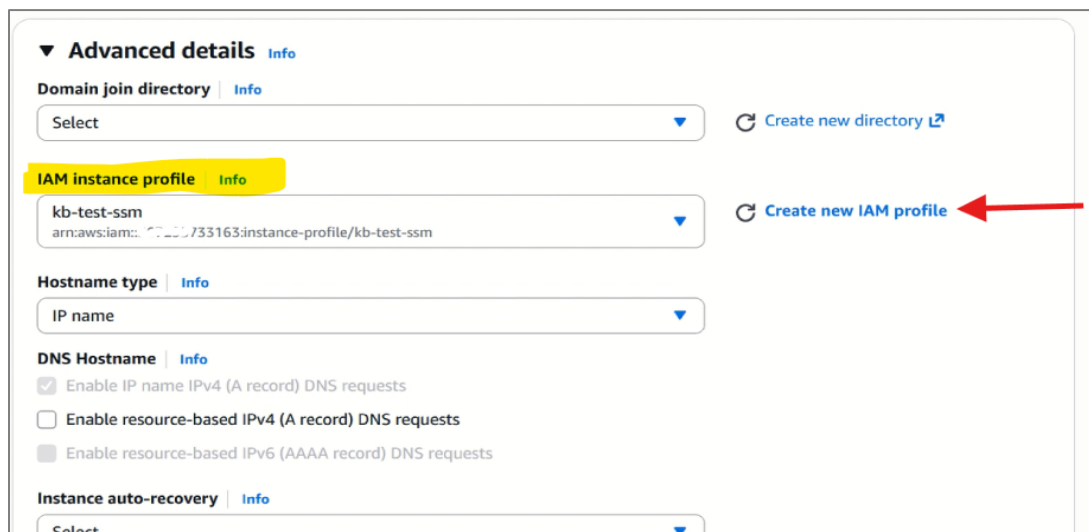
Cancel **Create key pair**

This step **creates a key pair** consisting of a **public key** and a **private key**). AWS stores the public key, and you will be prompted to save the private key on your local machine. Make sure to remember where you save the private key. In the future, the public key will appear in the AWS drop-down list.



For SSM Session Manager-based connection:

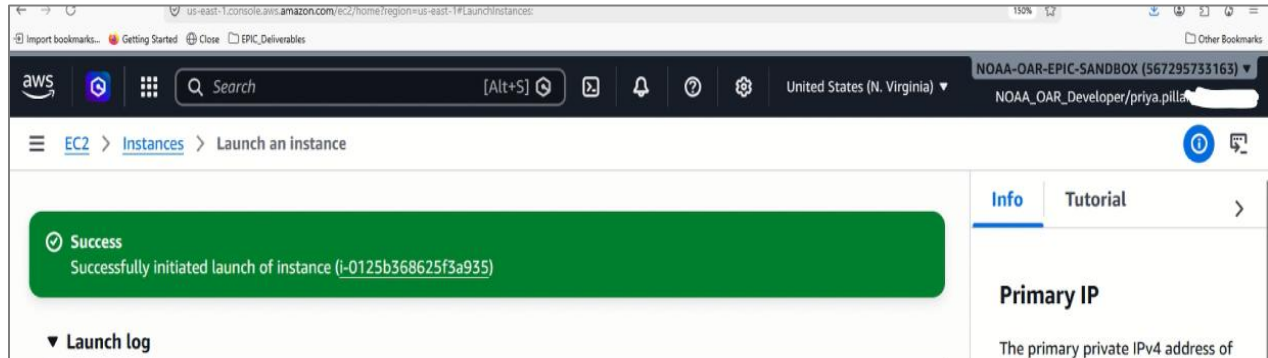
You do **not** need to select a key pair, and **SSH** should be disabled. You must attach an **IAM instance profile** that has SSM permissions. You may use an existing profile, or create a new IAM profile if one is not already available



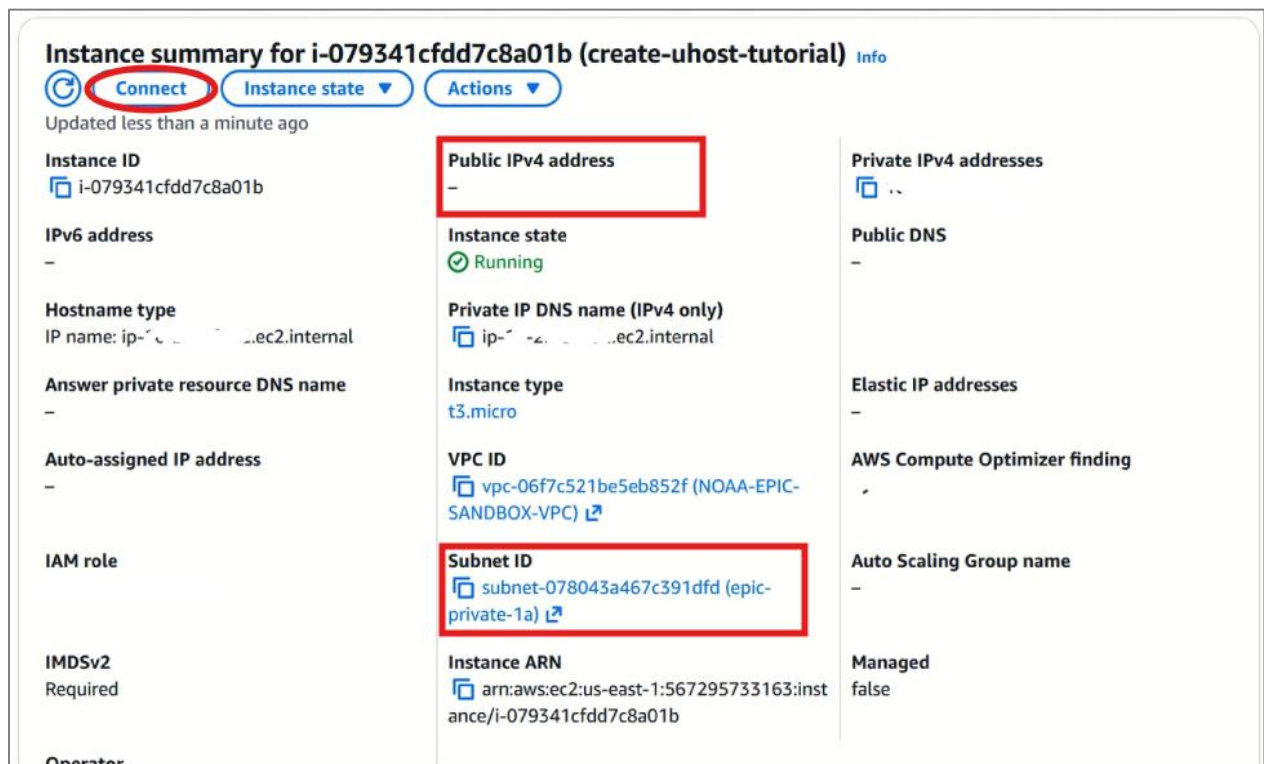
- **Network settings:** Leave as defaults
- **Configure storage**
 - 1x 20 GiB gp3
 - File systems: None
- **Advanced details:** Leave as defaults

Launch the Utility host (i.e., AWS EC2 Linux instance) you just created

Once you have filled in all the required settings, click the **“Launch instance”** button on the right sidebar (or at the bottom, if using a narrow window) to start the **EC2 instance**. You will then see a success message displaying the **instance ID**, which links directly to the instance page.



Click on the **“instance ID”** to open the instance page, which will display the **public IP** address for the instance. The example shown here uses a different “instance” than the one I created in the **“Build a Utility host”** demo, as it was provided by my cloud admin. Make sure to note the **Public IPv4 address** and the **Subnet ID** of your instance, we will need them in the next step.



Connect to the Utility host using SSM Session manager

Click the “Connect” button on the “**create-uhost-tutorial**” Instance to connect to the utility host and proceed with building the cluster image

The screenshot shows the AWS Management Console interface for connecting to an EC2 instance. The breadcrumb navigation indicates the path: EC2 > Instances > i-079341cfdd7c8a01b > Connect to instance. The instance details section shows the Instance ID (i-079341cfdd7c8a01b), VPC ID (vpc-06f7c521be5eb852f), Security groups (sg-0fdf9122b29fc9064), and IAM role. Below this, the 'EC2 Instance Connect' tab is active, and the 'SSM Session Manager' sub-tab is selected and circled in red. The 'SSM agent status' section displays the following information:

SSM agent status			
Ping status Online	Last ping time Wed Jun 24 2026 19:02:50 GMT-0400 (Eastern Daylight Time)	Session Manager connection status Connected	Agent version 3.3.4624.0

Below the status section, the 'Session Manager usage:' section provides instructions:

- Connect to your instance without SSH keys, a bastion host, or opening any inbound ports.
- Sessions are secured using an AWS Key Management Service key.
- You can log session commands and details in an Amazon S3 bucket or CloudWatch Logs log group.
- Configure sessions on the Session Manager [Preferences](#) page.

At the bottom right of the console, there are two buttons: 'Cancel' and 'Connect'. The 'Connect' button is circled in red.

2. Build Utilities and Amazon Machine Image (AMI)

From the “**create-uhost-tutorial**” instance, click on SSM Session Manager connection type and click “connect”. From here users can proceed with building the Packer and with that packer, build the AWS cluster image (i.e, AMI) for Land DA. This AMI will contain the OS libraries, MPI, Python packages, and dependencies needed for Land DA.

Connect to Utility host. Users can use any Linux OS based host.

If your utility host is Amazon Linux 2 based

```
>> sudo su - ec2-user
```

If your utility host is Ubuntu based

```
>> sudo su - ubuntu
```

Copy “**build_utility.sh**” from [aws-landda-tutorial](https://github.com/NOAA-EPIC/aws-landda-tutorial) GitHub Repository

Copy the packer build script, “**build-utility.sh**” from [aws-landda-tutorial](https://github.com/NOAA-EPIC/aws-landda-tutorial) repo

```
>> wget https://github.com/NOAA-EPIC/aws-landda-tutorial/build-utility.sh
```

The next step is to run the packer build script, **build_utility.sh**

```
>> sudo ./build-utility.sh
```

Clone AWS ParallelCluster build scripts to build the Land DA v3 AWS cluster

```
>> git clone https://github.com/NOAA-EPIC/aws-landda-tutorial.git
```

```
>> cd aws-landda-tutorial
```

Use “screen” utility to run computationally intensive commands in the background

```
>> which screen
```

If you don’t find screen installed, run

```
>> sudo yum install screen
```

And then run

```
>> screen
```


Sets the environment variable AWS_REGION to **us-east-1**

```
>> export AWS_REGION=us-east-1
```

Disable colored output to avoid color codes clutter terminal logs

```
>> export PACKER_NO_COLOR=1
```

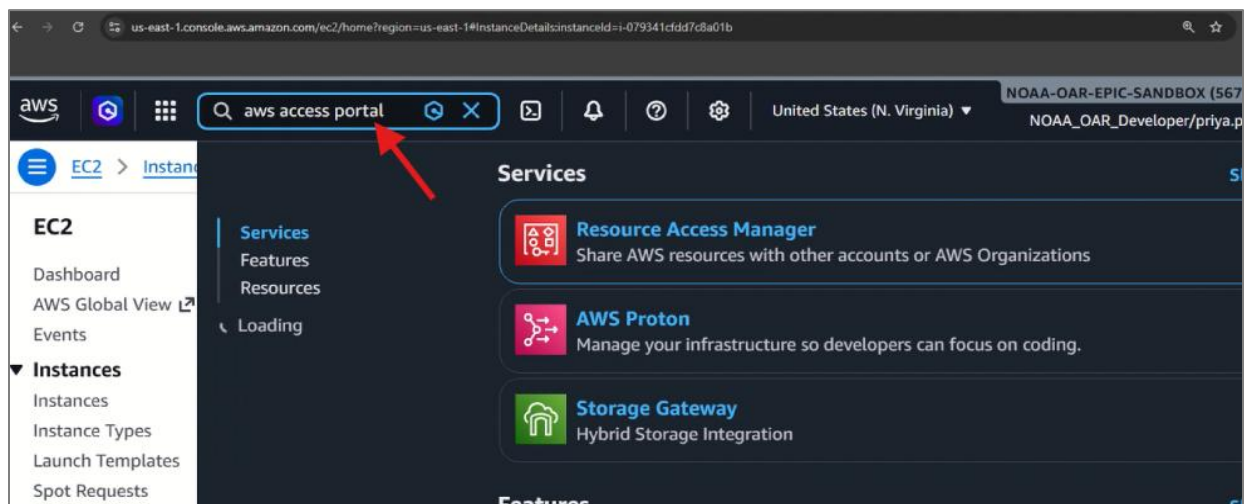
Enables **debug logging** for Packer and specifies the file where Packer should write its logs

```
>> export PACKER_LOG=1
```

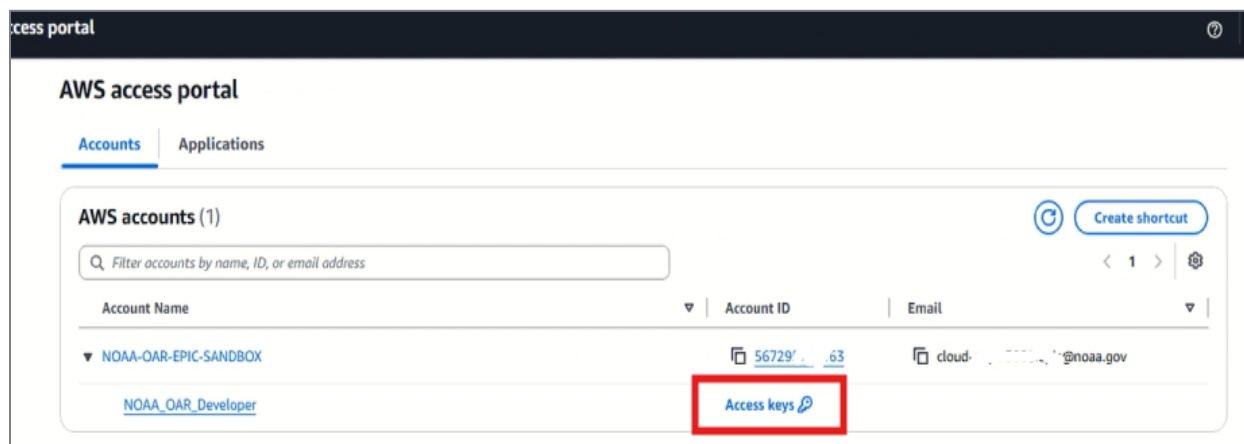
```
>> export PACKER_LOG_PATH="packer.log"
```

Copy AWS Environment variable

Go to AWS console, search for “**AWS Access Portal**”



Users will find a screen similar to the following and click on “**Access keys**”



Copy the “**AWS environment variables**” using the copy button adjacent to the access keys on the access key page. **Please note that these variables are Sensitive and NEVER SHARE them with anybody.**

These variables are:

AWS_ACCESS_KEY_ID

AWS_SECRET_ACCESS_KEY &

AWS_SESSION_TOKEN

Get credentials for NOAA_OAR_Developer

Create access for the account NOAA-OAR-EPIC-SANDBOX (567295733163) with NOAA_OAR_Developer. Use any of the following options to access AWS resources programmatically or from the AWS CLI. You can retrieve credentials as often as needed.

macOS and Linux | Windows | PowerShell

▼ **AWS IAM Identity Center credentials (Recommended)**

To extend the duration of your credentials, we recommend you configure the AWS CLI to retrieve them automatically using the [aws configure sso](#) command. [Learn more](#)

SSO start URL:

SSO Region:

▼ **Option 1: Set AWS environment variables**

Run the following commands in your terminal to set the AWS environment variables. [Learn more](#)

```
export AWS_ACCESS_KEY_ID=
export AWS_SECRET_ACCESS_KEY=
export AWS_SESSION_TOKEN=
```

▼ **Option 2: Add a profile to your AWS credentials file**

Copy and paste the following text in your AWS credentials file (~/.aws/credentials). [Learn more](#)

```
[567295733163_NOAA_OAR_Developer]
aws_access_key_id=
aws_secret_access_key=
aws_session_token=
```

▼ **Option 3: Use individual values in your AWS service client**

Copy and paste these values into your code. [Learn more](#)

AWS access key ID:

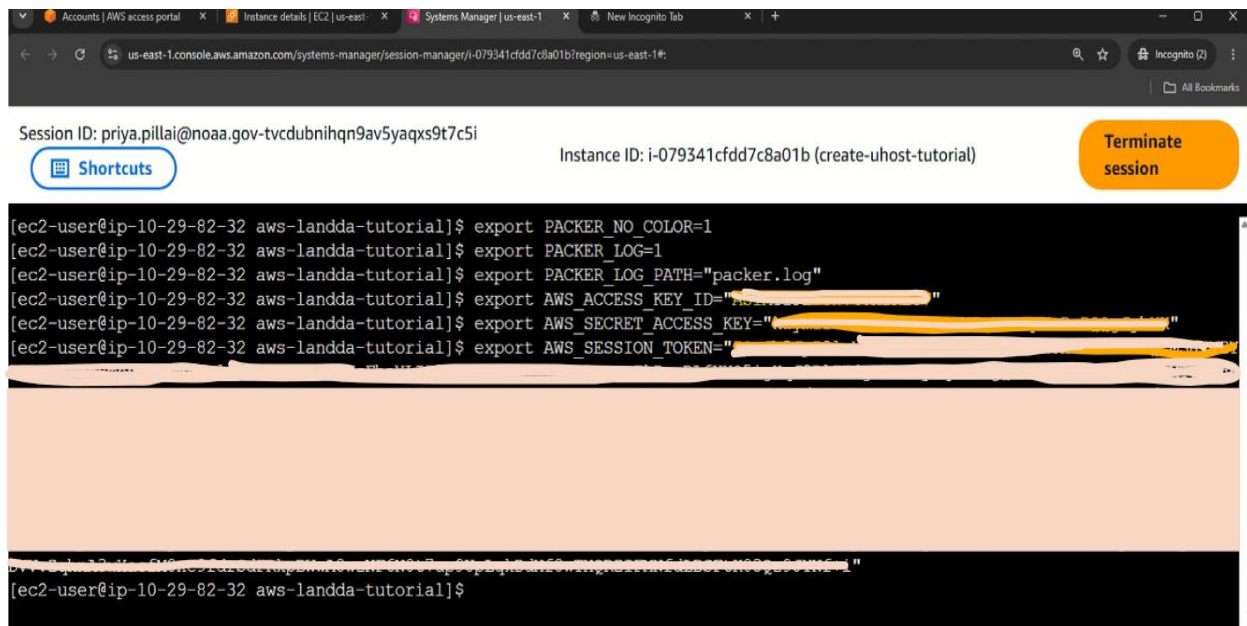
Export the AWS Environment variables to the terminal

Paste the **AWS ACCESS Key** credentials you copied to the terminal and it will automatically paste those variables one by one. Once again, these variables are Sensitive and NEVER SHARE them with anybody.

```
>> export AWS_ACCESS_KEY_ID = < your AWS variable>
```

```
>> export AWS_SECRET_ACCESS_KEY = < your AWS variable>
```

```
>> export AWS_SESSION_TOKEN = < your AWS variable>
```



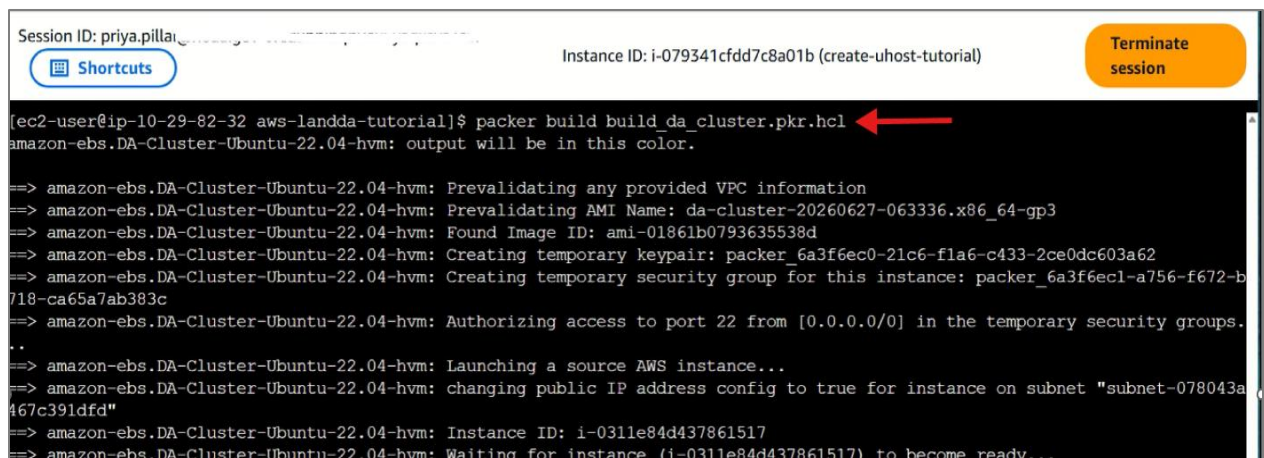
Build the packer and AWS Cluster Image for Land DA v3.0

Open "**build_da_cluster.pkr.hcl**" and update **subnet** field with the **subnet ID** of your utility host

```
>> vi build_da_cluster.pkr.hcl
```

You are now ready to build the initial baseline image. Run "**build_da_cluster.pkr.hcl**"

```
>> packer build build_da_cluster.pkr.hcl
```



```
Session ID: priya.pillai Instance ID: i-079341cddd7c8a01b (create-uhost-tutorial) [Shortcuts] [Terminate session]

[ec2-user@ip-10-29-82-32 aws-landda-tutorial]$ packer build build_da_cluster.pkr.hcl
amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: output will be in this color.

==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: Prevalidating any provided VPC information
==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: Prevalidating AMI Name: da-cluster-20260627-063336.x86_64-gp3
==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: Found Image ID: ami-01861b0793635538d
==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: Creating temporary keypair: packer_6a3f6ec0-21c6-fla6-c433-2ce0dc603a62
==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: Creating temporary security group for this instance: packer_6a3f6ec1-a756-f672-b718-ca65a7ab383c
==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: Authorizing access to port 22 from [0.0.0.0/0] in the temporary security groups.
..
==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: Launching a source AWS instance...
==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: changing public IP address config to true for instance on subnet "subnet-078043a467c391dfd"
==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: Instance ID: i-0311e84d437861517
==> amazon-ebs.DA-Cluster-Ubuntu-22.04-hvm: Waiting for instance (i-0311e84d437861517) to become ready...
```

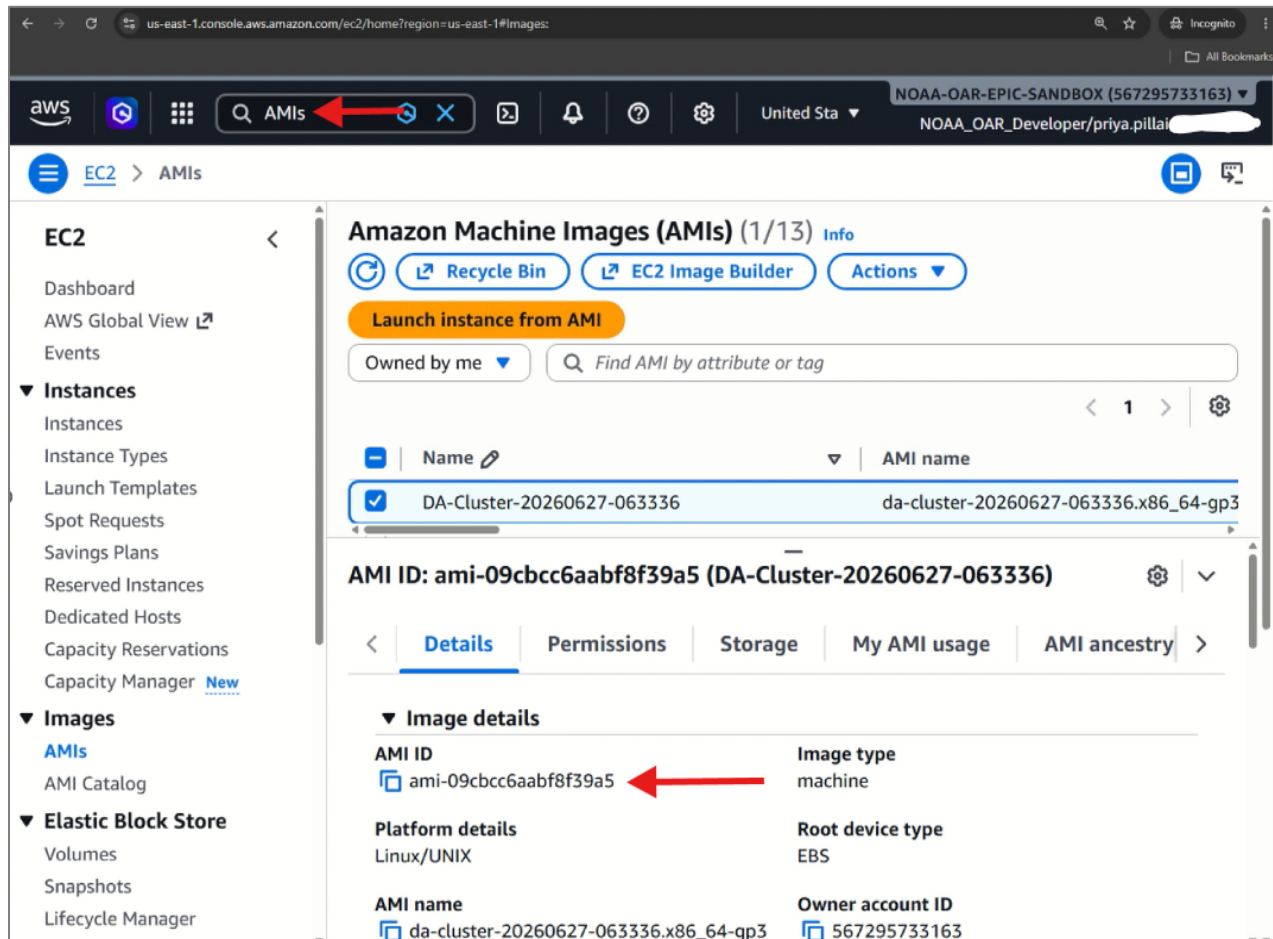
You can detach from the screen using the **Ctrl + (A followed by D)**, and the Packer build script will continue running in the background even after you terminate (i.e., close) the session. It takes **about 3 hours to build the cluster image**. Follow the progress through the **packer.log** file.

Once the build is complete, you will see this newly created AMI listed under **AWS AMIs**.

3. Spin up the AWS Parallel Cluster

Once the build is complete, you should see an AMI listed under your Amazon Machine Images. Take note of the **AMI ID**, as this will be used in the AWS ParallelCluster configuration.

For AMI id, go to AWS console, search for AMIs, click on the AMI you created for this tutorial and copy the AMI ID.



Now, open the cluster build script, **da_hpc.yaml** and update the following to match your instances credentials. You already have the **Subnet ID** and (**Keypair name**, if using SSH). Now on the terminal,

```
>> vi da_hpc.yaml
```

- **CustomAmi**
- **SubnetId** (multiple places in this script)
- **KeyName**

```
Session ID: priya.pillai... Instance ID: i-079341cddd7c8a01b (create-uhost-tutorial)
t3q Shortcuts Terminate session

Region: us-east-1
Image:
  Os: ubuntu2204
  CustomAmi: <your AMI ID here>
HeadNode:
  InstanceType: c7i.2xlarge
  Networking:
    SubnetId: <your subnet ID here>
  Ssh:
    KeyName: <your SSH key name>
  LocalStorage:
    RootVolume:
      Size: 500
      VolumeType: gp3
      Iops: 10000
      Throughput: 1000
  Iam:
    AdditionalIamPolicies:
      - Policy: arn:aws:iam::aws:policy/AmazonS3FullAccess
      - Policy: arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore
Scheduling:
  Scheduler: slurm
  SlurmQueues:
    - Name: compute
      ComputeResources:
        - Name: c7i24xlarge
          Instances:
            - InstanceType: c7i.24xlarge
              MinCount: 0
              MaxCount: 2
          Efa:
            Enabled: false
          Networking:
```

You already have the **Subnet ID** and **Keypair** name. Now on the terminal, run the following command to create the initial cluster.

```
>> pcluster create-cluster --region us-east-1 --cluster-name
landda-tutorial-cluster --cluster-configuration da_hpc.yaml
```

For additional information refer to the [AWS ParallelCluster documentation](#)

This process should take about **20 minutes** to complete. You can view the status of your cluster through the **AWS ParallelCluster CLI tool** by issuing the following command:

```
>> pcluster list-clusters --region us-east-1
```

The results should list the cluster status as **CREATE_COMPLETE** once the operation has successfully completed.

```
}
},
{
  "clusterName": "Your cluster name",
  "cloudformationStackStatus": "CREATE_COMPLETE",
  "cloudformationStackArn": "arn:aws:cloudformation:us-east-1:56",
  "region": "us-east-1",
  "version": "3.13.2",
  "clusterStatus": "CREATE_COMPLETE",
  "scheduler": {
    "type": "slurm"
  }
}
```